# Mandate of Three — Account Economy & Marketplace

**Specyfikacja produktowo-techniczna v1.1 · 13 października 2026**
**Status:** projekt do implementacji etapami; reguły oznaczone jako „decyzja” są przyjętymi założeniami, parametry liczbowe są propozycjami do testów.
**Zakres:** portfel Yang na konto, Player Shops, wyszukiwarka w duchu oficjalnego Szkła Handlu z filtrami bonusów PoE, Currency Exchange / bulk trade, przyszłe kanały i zamknięcie sezonu.
**Podstawa techniczna:** `main` repozytorium `PatrykCieszynski/Mandate-of-Three`, SHA `e362b9ebcc9bb73f591cc9a-99542b9d16c1c3048`.

> **TL;DR:** Jedna baza SQLite na logiczny World. Seasonal uruchamiamy teraz; Classic kiedyś później. W Worldzie: jeden portfel Yang i jeden prywatny sklep na konto, wspólny rynek dla wszystkich map, dungeonów i przyszłych kanałów. Unikalne przedmioty sprzedajemy z siatki sklepów; waluty/materiały wymieniamy przez odrębny Currency Exchange z automatycznym dopasowaniem zleceń. Wszystkie operacje ekonomiczne są autorytatywne, trwałe, atomowe i odporne na ponawianie.

SPIS TREŚCI


1. Decyzje i granice
2. Cele i doświadczenie gracza
3. UX i rzeczywiste referencje
4. Baza kodu i zależności wdrożenia
5. Migracja Yang: postać → konto
6. Model Player Shops
7. Wyszukiwarka i otwieranie sklepu
8. Currency Exchange — PoE-style bulk trade
9. Proponowany model tabel (logiczny)
10. Kontrakty serwer–klient i granice uprawnień
11. Integracja Core UI / CEF
12. Durability, kopie i awarie
13. Seasonal → Classic: kontrakt przyszłej migracji
14. Testy akceptacyjne
15. Plan implementacji i kryteria wyjścia
16. Pytania designerskie, które mogą poczekać
17. Referencje repozytorium i UX


## 1. Decyzje i granice

| Temat | Ustalenie |
|---|---|
| Pierwszy World | **Seasonal**; na teraz jedyny rozwijany świat |
| Drugi World | **Classic**; uruchomienie później, być może po pierwszym sezonie |
| Baza | Osobne SQLite dla Seasonal i Classic; jedna baza na logiczny World |
| Po sezonie | Postacie i ich majątek z Seasonal **trafiają do Classic**, z kontrolowaną procedurą scalania |

| Temat | Ustalenie |
|---|---|
| Kanały / warstwy | Niezdecydowane i poza obecnym zakresem; nie mogą dzielić rynku na części |
| Yang | Jeden portfel na konto **w danym Worldzie**; wszystkie postacie mają dostęp |
| Sklep | Jeden trwały Player Shop na konto **w danym Worldzie** |
| Dostępność | Offline, niezależnie od mapy, instancji i zalogowania właściciela |
| Sprzedaż unikalnych przedmiotów | Własny sklep z fizyczną siatką i ceną całej pozycji w Yang |
| Wyszukiwarka | Wszystkie sklepy danego Worlda; wybór oferty otwiera **cały sklep**, podświetlając wynik |
| Currency Exchange | Osobny handel fungible w stylu PoE: zlecenia, kurs, płynność, częściowe wykonania |
| Fizyczne sklepy M1 | Późniejsza prezentacja kilku rotujących stoisk, bez tworzenia osobnych rynków |
| Ulubieni sprzedawcy / reputacja | **Nie robimy** |
| PostgreSQL | **Nie teraz**; osobny etap, jeśli pojawi się wiele procesów zapisujących wspólną ekonomię |

**Nie mylić:** `World` jest jednostką trwałości gospodarki i postaci; `Instance` jest uruchomioną kopią mapy/dungeonu; `Channel` / `Layer` będzie sposobem dzielenia populacji wewnątrz jednego Worlda. Identyfikator instancji **nigdy** nie wchodzi do kluczy własności sklepu, portfela ani oferty.

### Poza zakresem pierwszego wdrożenia

Nie budujemy teraz wielu World Serverów zapisujących jedną bazę, cross-world marketplace'u, automatycznego layerowania, gildyjnych sklepów, prowizji, aukcji z licytacją, reputacji sklepów, ofert pożyczkowych ani market-data API dla stron trzecich. Przeniesienie Seasonal → Classic opisujemy i testujemy jako przyszły etap, ale **nie** uruchamiamy fikcyjnego Classic tylko dla tej funkcjonalności.

## 2. Cele i doświadczenie gracza

Gracz zbiera Yang jedną postacią, wydaje dowolną inną na tym samym koncie i Worldzie. Ma jeden trwały sklep, do którego układa przedmioty przeciąganiem ze swojego ekwipunku. Ustala ceny; po potwierdzeniu przedmioty opuszczają ekwipunek i przechodzą pod opiekę sklepu (escrow). Nie potrzeba stojącego w M1 altsklepu ani uruchomionego klienta.

Kupujący wybiera bazowy typ przedmiotu (bez poziomu +N w nazwie), zakres ulepszenia od +0 do +9, wymagane bonusy i cenę; następnie wybiera wynik i otwiera **cały sklep sprzedającego**, widzi zaznaczoną pozycję i może obejrzeć inne przedmioty. Kupuje bez kontaktu ze sprzedającym; transakcja jest natychmiastowa. Przy normalnej sprzedaży Yang od razu zasila portfel konta sprzedającego, także offline.

Materiały i waluty fungible nie zapychają siatki sklepu: Currency Exchange ma własny ekran, kursy, zlecenia kupna i sprzedaży oraz automatyczne częściowe rozliczenia. Celem jest oszczędzanie czasu przy bulk trade, a nie naśladowanie nieefektywnej wymiany wiadomościami.

### Model interakcji Player Shops

1. Otwórz **Mój sklep**, ustaw nazwę, przeciągnij wybrany przedmiot z własnego ekwipunku na wolne miejsce siatki.

2. Wpisz cenę całej pozycji w Yang i zatwierdź. Serwer atomowo przenosi konkretny `ItemInstance` do escrow sklepu; wystawienie staje się widoczne w wyszukiwarce.
3. Po wylogowaniu sklep pozostaje widoczny i można w nim kupować. Przedmioty w escrow nie mogą zostać użyte, wyposażone, ulepszone, zdeponowane ani sprzedane ponownie.
4. Edytuj cenę, przesuń ofertę w siatce, anuluj pozycję lub wstrzymaj cały sklep. Anulowany przedmiot wraca do kontowego odbioru; odbiór do postaci to oddzielna operacja z weryfikacją miejsca.
5. Przy zakupie serwer ponownie sprawdza aktualną ofertę, cenę, saldo i pojemność ekwipunku. Kupujący otrzymuje przedmiot, sprzedający Yang, a oferta znika — w **jednym COMMIT**.

**Decyzja UX:** siatka jest realnym układem sklepu (w tym wysokość przedmiotów 1–3 sloty jak w obecnym Inventory), nie tylko dekoracją listy ofert. Układ jest trwały. Nie wymagamy opcji „zapamiętaj sprzedawcę".

# 3. UX i rzeczywiste referencje

**Wiążąca referencja wizualna: Metin2 Wiki — Szkło Handlu: Informacje.** Oficjalna strona zawiera screenshoty panelu, podział filtrów i opis tabeli wyników. To główny materiał do obejrzenia przy implementacji, **nie** nasze wcześniejsze wygenerowane makiety. Screenshotów Metin2 nie przechowujemy w tej paczce lokalnie: pozostają dostępne w źródle i zachowują prawa wydawcy.

Wzorzec: kompaktowy, natywnie wyglądający interfejs MMORPG, osobna sekcja filtrów i gęsta, sortowalna tabela ofert. **Nie** kopiować dosłownie artworku, ikon i tekstur; wykorzystać Core UI Mandate. W wersji Mandate zakres wyszukiwania obejmuje **cały logiczny World**, nie tylko jeden CH; kliknięcie wyniku **otwiera cały sklep** z wyróżnioną pozycją, zamiast kupować bezpośrednio z wyników. Tooltip pokazuje pełne dane przedmiotu na hoverze, a na liście są jedynie informacje potrzebne do porównania.

## 3.1. Wyszukiwarka — docelowe zachowanie

| Region | Zawartość | Zasada |
|---|---|---|
| Góra / szybkie filtry | Kategoria, bazowa definicja lub nazwa, **upgrade OD / DO**, cena OD / DO | Widoczne stale, zwarte kontrolki |
| Filtry zaawansowane | Dodawanie warunków na konkretne bonusy, wartość minimalną/maksymalną | Domyślnie `AND` między bonusami; `OR` poza V1 |
| Wyniki | Ikona, nazwa i upgrade ( `Miecz +8` ), ilość, cena całkowita, opcjonalnie cena/szt., akcja `Otwórz sklep` | Gęsta lista / tabela; sortowanie; server-side page/cursor |
| Hover | Wspólny tooltip przedmiotu z Inventory (rolls/bonusy, sockety, statystyki) | Bez powielania pełnych statystyk w każdym wierszu |
| Wybór oferty | `open_shop(shop_id, highlight_offer_id)` | Pełna siatka danego sklepu, wyszukany UID wyróżniony |

**Jedna definicja, wiele poziomów ulepszenia — wymaganie V1:** wyszukiwarka katalogu proponuje `Miecz` **raz** jako `definition_id` , nigdy osobne pozycje `Miecz +0` , `Miecz +1` , … `Miecz +9` . Poziom jest cechą konkretnego `ItemInstance` , nie osobną definicją. Kontrolki `od +0` **i** `do +9` są niezależne od pola nazwy; domyślnie obejmują wszystkie poziomy. Ustawienie `+7..+9` znajduje egzemplarze +7, +8 i +9; `+9..+9` tylko +9. W UI dopuszczamy `od == do` ; gdy użytkownik ustawia granice w odwrotnej kolejności, **blokujemy wysłanie zapytania z komunikatem walidacji** (nie korygujemy tego po cichu).

To w istocie upraszcza obsługę funkcji, którą oficjalne Szkło Handlu już częściowo oferuje: osobny zakres *poziomu przedmiotu* od 0 do 9. U nas połączymy go z jednoznacznymi podpowiedziami bazowych nazw i filtrowaniem bonusów.

**Filtry bonusów w stylu PoE:** gracz wybiera bonus z listy metadanych istniejących w grze ( `stat_id` ), operator z dozwolonej listy i próg liczbowy z odpowiednią jednostką. Np. `silny_przeciw_potworom >= 10%` **AND** `szansa_krytyczna >= 5%` . W V1 filtrujemy *losowane bonusy instancji* ( `ItemInstance.affixes` ), a nie wyświetlany tekst ani sumę z bazowymi statystykami i ulepszeniem; osobne filtry statystyk końcowych zostawiamy na później. Nie dopuszczamy dowolnego `stat_id` /operatora z klienta. Przy powtarzającym się `stat_id` przynajmniej jeden pojedynczy wpis rolla musi spełniać próg (nie sumujemy dwóch rolli domyślnie). Warunki `AND` są niezależne i wszystkie muszą się zgadzać.

**Nazwy:** wyszukiwanie po fragmencie nazwy jest przyjazne polskim znakom i wielkości liter. Typeahead zwraca **unikalne bazowe definicje**, z kontrolowanego katalogu serwera; wybór zapisuje `definition_id` , a nie lokalizowany napis. Zmiana języka UI nie zmienia filtrowania. Jeżeli użytkownik nie wybierze konkretnej definicji, dopuszczamy wyszukiwanie tekstowe po indeksowanych nazwach (bez ukrytego parsowania suffixu `+9` ).

### 3.2. Schemat przepływu (wireflow, nie grafika docelowa)

```
KATALOG:  [Miecz v]        ULEPSZENIE:  [+0 v]  ...  [+9 v]
BONUSY:   [Potwory >= 10%] [Krytyk >= 5%]      [ + dodaj bonus ]
CENA:     [od Yang]  ...  [do Yang]              [ SZUKAJ ]
-------------------------------------------------------------
PRZEDMIOT          ILOŚĆ       CENA          AKCJA
Miecz +9           1           15 000 000    Otwórz sklep
Miecz +8           1            9 500 000    Otwórz sklep
-------------------------------------------------------------
HOVER: wspólny tooltip    KLIK: cały sklep + podświetlona oferta
```

To jest wyłącznie opis układu i akcji. Wygląd i proporcje elementów powinny być dobrane w Core UI w kontekście rozdzielczości gry oraz obsługi myszy.

### 3.3. Player Shop — ta sama siatka, inne role

Po otwarciu z wyników użytkownik widzi cały sklep i podświetloną ofertę. Właściciel ma siatkę w trybie edycji, drag & drop z Inventory, pola ceny i akcje zmiany/wycofania; odwiedzający — widok tylko do odczytu oraz zakup. Nazwa sklepu i położenie slotów pozostają trwałe. Nie planujemy ulubionych sprzedawców ani systemu reputacji.

### 3.4. Currency Exchange — osobny UX w stylu Path of Exile

To **nie jest drugi widok wyszukiwarki itemów**. Gracz wybiera parę aktywów `mam / chcę` , ilość, kurs limit, widzi głębokość rynku, `free/reserved` oraz wyniki częściowych realizacji. Zlecenia przechodzą przez osobny ledger i orderbook (sekcja 8). Waluty nie zajmują slotów Player Shop.

### 3.5. Linki referencyjne

- **Oficjalne Szkło Handlu — pełny opis i screenshoty** (główna referencja, Gameforge).
- MarsisMt2 — projekt UI Metin2 na Behance (dodatkowa inspiracja stylistyczna).
- Ikarus — rework Offline Shop Search (dodatkowa inspiracja stylistyczna).
- Path of Exile — oficjalna gra (inspiracja filtrami statystyk i osobnym systemem bulk exchange; zachowanie szczegółowo opisane tu jako *nasza* decyzja).

Wszystkie odnośniki wskazują oryginalne materiały; zrzuty ekranów nie są twórczością Mandate. W repozytorium przechowujemy własne kontrakty UX, a **nie cudze grafiki**. Poprzednie wygenerowane mockupy z v1.0 nie są już referencją docelową.

## 4. Baza kodu i zależności wdrożenia

Sprawdzone w repozytorium:

| Obszar | Obecnie | Zmiana |
|---|---|---|
| `WalletStoreSqlite` | `wallets(character_id, yang)` ; max 9 × 10^15 | `account_wallets(account_id, yang)` ; nowy owner |
| `WorldDatabase` | RAM/pending/dirty wallet indeksowane `player_id` | indeks `account_id` ; jedna kopia RAM na konto |
| `CurrencyEndpoint3D` | `_owner(peer)` zwraca `player_id` | trwała tożsamość konta przez zaufaną sesję |
| `WorldStoreSqlite.create_player_character` | nowa postać dostaje portfel/starting Yang | brak drugiego grantu na alt; pierwszy portfel konta tylko raz |
| `ShopStoreSqlite` / `UpgradeStoreSqlite` | rozliczenie portfela postaci w transakcji | wspólny portfel konta, zachowana atomowość |
| `ItemInstance` | UID, owner_character_id, revision, amount, bonusy/sockety | bez resetowania UID; escrow i wyłączność umiejscowienia |
| `item_placements` | tylko `bag` / `equipment` | jednoznaczna lokalizacja shop/claim bez zdublowania UID |
| `AccountStorageSqlite` | dzielony `account_name` , osobna lokacja | wspólna tożsamość konta i wyłączność umiejscowienia |
| Master | `AccountResource.id` , `username` ; zapis w `.tres` | stabilny `account_id` w zaufanym handoff do Worlda |
| `CoreUiController` i TypeScript | inventory/equipment/storage/NPC shop/upgrade | ekrany Search, Shop, Exchange i nowe kontrakty IPC |

**Istotny niuans identyfikacji:** `WorldSchema.accounts` i Account Storage aktualnie używają `account_name` , ale Master ma już liczbowy `AccountResource.id` . Preferowany trwały klucz nowej ekonomii to **zweryfikowany** `account_id` **z Mastera**, przekazany serwer-serwer w handoff. Nazwa konta to atrybut, nie klucz ekonomii. Migracja starych kont bez wiarygodnego mapowania ID musi użyć jawnego mapowania z Mastera, nigdy zgadywania ani argumentu RPC klienta.

### Uwaga o istniejącym „Trade”

Repo zawiera odziedziczone `TradeService` , `TradeTable` i `shop_trade.gd` dla handlu/stołów 2D. To **nie jest** obecny system offline Player Shops/Currency Exchange. Nie należy podłączać nowego rynku do starego systemu tylko dlatego, że nazwy się pokrywają.

## 5. Migracja Yang: postać → konto

### 5.1. Semantyka

- `account_wallets(account_id)` jest jedynym źródłem Yang wewnątrz danego Worlda.
- Wszystkie postacie tego konta widzą ten sam `balance` . Nie istnieje osobny „portfel sklepu”.
- Shop sale natychmiast uznaje portfel konta sprzedającego, również offline.
- Exchange rezerwuje ofertowane Yang w escrow zlecenia; nie można nimi płacić ponownie, dopóki zlecenie jest aktywne.

- Grindowanie Yang nadal może korzystać z RAM i ~30-sekundowego checkpointu; ryzyko utraty *ostatnich niezapisanych dropów po awarii* pozostaje jawne jak obecnie. Operacje handlowe nie mogą z tego powodu utracić ani zdublować Yang.
- Yang nigdy nie są przenoszone między Worldami w trakcie sezonu. Dopiero sezonowa migracja zasila Classic.

### 5.2. Migracja danych

1. Zrób poprawny backup SQLite razem ze stanem WAL i zatrzymaj nowe operacje ekonomiczne.
2. Zapewnij mapowanie wszystkich `players.account_name` → **zaufany** `account_id` Mastera; odrzuć osierocone/mieszane rekordy.
3. Załóż portfel kontowy i sumuj **istniejące salda SQL wszystkich postaci** tego konta **dokładnie raz**. Zastosuj bezpieczny limit i jawny raport overflow, bez obcinania kwot.
4. Zamknij/rozlicz pending RAM przed migracją; migracja na zatrzymanym Worldzie jest obowiązkowa. Nie sumuj ponownie tych samych delta-checkpointów.
5. Zapisz numer migracji / receipts. Przed zatwierdzeniem sprawdź sumę globalną Yang, liczbę kont i brak ujemnych sald; po migracji te same asercje.
6. Przełącz wszystkie ścieżki: loot Yang, NPC Shop, Upgrade, HUD, logout, shutdown, dirty wallet, test wydatków. Usuń stare ścieżki `character_id`, nie zostaw dwóch portfeli równocześnie.
7. Nowy alt nie dostaje ponownie `STARTING_YANG`; seed jednorazowy przy pierwszym utworzeniu portfela danego konta/Worlda, niezależnie od tworzenia postaci.

**Niebezpieczny edge case:** w tej samej sekundzie konto otrzymuje 100 Yang z lootu (RAM pending), kupuje coś za 70 Yang (SQL) i sprzedaje offline miecz za 200 Yang (SQL). Wynik ma być zawsze taki sam, niezależnie od kolejności RPC i checkpointów; po COMMIT RAM musi pokazywać pełne saldo.

### 5.3. Reguła transakcyjna portfela

`account_yang = persisted + pending_income` to autorytatywny widok przy aktywnym koncie w jednym procesie. Operacja krytyczna w `BEGIN IMMEDIATE` najpierw aplikuje pending income **wewnątrz tej samej transakcji**, potem wydaje/uznaje środki i modyfikuje przedmioty. Dopiero po COMMIT aktualizuje cache RAM oraz zeruje zmaterializowany pending. Sprzedaż dla offline konta modyfikuje tylko SQLite. Dla online sprzedającego odświeża lub koryguje jego **jedną** kopię RAM, bez nadpisania nowszego pending income.

Warunek implementacyjny: wszystkie modyfikacje Yang — NPC, upgrade, loot, shop, exchange, komendy administracyjne — przechodzą przez **jedno jasno określone API ekonomii**, nie przez niezależne `UPDATE wallets` rozrzucone w domenach. Nie wymaga to ogólnego ORM ani migracji na PostgreSQL.

## 6. Model Player Shops

### 6.1. Podstawowe jednostki

- `AccountShop`: `shop_id`, `account_id`, `display_name`, `status` (`active|paused`), `revision`, `created_at`, `updated_at`. Unikalność: **UNIQUE(account_id)** w obrębie bazy Worlda.
- `ShopOffer`: `offer_id`, `shop_id`, `item_uid`, `grid_position`, `price_yang`, `revision`, `created_at`. `UNIQUE(item_uid)`, `UNIQUE(shop_id, grid_position)` po uwzględnieniu footprintu wielokomórkowego.
- `ShopEscrow`: trwała relacja powierzenia konkretnego UID kontu i sklepowi; pozycja w siatce/offer ID powiązane jednoznacznie.
- `AccountClaims`: miejsce odbioru anulowanych ofert i innych bezpiecznych dostaw, niezależne od pełnego ekwipunku postaci. Claim nie jest wystawioną ofertą.
- `EconomyReceipt`: idempotentny rezultat krytycznej operacji i zapis jej identyfikatorów dla audytu.

`item_instances.owner_character_id` może pozostać **provenance** pierwotnej postaci aż do legalnego odbioru/zakupu, ale nie może być podstawą uprawnień do itemu w shop/claim. Właściciel ekonomiczny itemu w escrow to konto wskazane w trwałej lokalizacji. Po zakupie `owner_character_id` zmienia się na ID otrzymującej postaci.

## 6.2. Wyłączność lokalizacji

Dla każdego `item_uid` istnieje **dokładnie jedna** legalna lokalizacja: `bag`, `equipment`, `account_storage`, `shop_escrow` albo `account_claim`. Nigdy dwa naraz. Aktualny model ma osobne `item_placements` i `account_storage`; dodanie shop wymaga albo jednego kanonicznego indeksu custody, albo wyraźnych triggerów/wspólnego kodu SQL broniącego wszystkich lokacji. **Testy mają wymuszać wyłączność na poziomie bazy**, a nie wyłącznie UI.

Wystawienie sprawdza: zaufane konto i postać, UID + revision, lokalizacja `bag`, prawa handlu itemem, kompletność footprintu oraz brak oferty. Następnie w transakcji usuwa pozycję z bag, umieszcza item w escrow, tworzy ofertę i zwiększa revision.

Anulowanie usuwa ofertę i przenosi UID do `account_claim`. Nie zwraca automatycznie do ekwipunku konkretnego alta; odebrać może dowolna postać tego konta z miejscem w bag. Pause sklepu nie przenosi itemów.

## 6.3. Reguły sklepu

- Jeden sklep na konto i World. Wszystkie postacie tego konta edytują ten sam sklep.
- Wystawienie nie wymaga bycia w M1 ani stałego połączenia po zakończeniu transakcji.
- Oferta nie wygasa automatycznie w V1. Sklep może być wstrzymany bez utraty escrow.
- Cena przechowywana jako dodatnia liczba całkowita w Yang; zakup całej pozycji, nawet gdy `amount > 1`. Handel częściami stosu odbywa się w Exchange.
- Nie można kupić ze **swojego** sklepu. Nie można wystawić wyposażonego/zablokowanego itemu, itemu specjalnego niehandlowalnego ani waluty z już zarezerwowanego Exchange.
- Zmiana ceny, usunięcie, przesunięcie i zakup wymagają `expected_revision` / warunku wersji, tak aby wyścigi kończyły się `stale` zamiast cichą utratą danych.
- Maksymalna pojemność na start **proponowana:** 32 pozycyjne sloty ofert z wielokomórkowym footprintem, rozszerzenie za decyzją balancingową. W implementacji limit ma być jedną konfiguracją Worlda, nie stałą rozsianą po UI/DB.

## 6.4. Zakup — przebieg autorytatywny

1. Klient przesyła `command_id`, `offer_id`, `expected_offer_revision`, `requested_bag_position` (opcjonalnie). **Nie przesyła ceny jako autorytetu**.
2. World identyfikuje kupującego z sesji; odczytuje konto, docelową postać i bieżącą ofertę pod blokadą transakcyjną (`BEGIN IMMEDIATE`).
3. Serwer sprawdza `active`, brak self-buy, revision, item w escrow, legalny typ, cenę oraz wolny footprint w bag kupującego. Jeśli brak miejsca: **brak zakupu i brak pobrania Yang**.
4. W tej samej transakcji: materializuje ewentualny pending Yang kupującego, obciąża jego kontowy portfel, uznaje kontowy portfel sprzedającego, zmienia właściciela i lokalizację UID, usuwa ofertę, zapisuje receipt/command_id.
5. **COMMIT** jest jedynym punktem przejścia własności. Po nim przygotuj autorytatywne cache/equipment/wallet i dopiero wyślij fakty/snapshoty do klienta.
6. Powtórzenie tego samego `command_id` zwraca zapisany wynik bez drugiego zakupu. Nowy `command_id` dla sprzedanej oferty daje `unavailable`.

Wariant dla przyszłości: kupno prosto do `account_claim`. Nie włączać w V1 bez decyzji UX — w pierwszym wdrożeniu zakup trafia do inventory albo kończy się bez kosztu, jeśli jest pełne.

# 7. Wyszukiwarka i otwieranie sklepu

**Źródło prawdy:** `ShopOffer` + `ItemInstance` w SQLite; katalog bazowych definicji i ich stat metadata to zaufany content. Wyszukiwanie jest projekcją tylko do odczytu. Lista może się zdezaktualizować między wyszukaniem a zakupem; **wyłącznie purchase** rozstrzyga własność.

## 7.1. Kontrakt zapytania V1

```
MarketSearchQuery
  definition_id?: String       # bazowy typ itemu (nie nazwa z '+N')
  name_query?: String          # wyszukiwanie tekstowe, jeśli brak definition_id
  category_id?: String         # tylko gdy istnieje w metadanych contentu
  upgrade_min: int = 0         # inkluzywnie
  upgrade_max: int = 9         # inkluzywnie
  affix_filters: [             # AND (V1), max wg konfiguracji
    { stat_id: String, op: 'gte'|'lte'|'eq', value: Number }
  ]
  price_min_yang?: int
  price_max_yang?: int
  sort: 'price_asc'|'price_desc'|'newest'
  cursor?: String
  limit: int                   # z limitem po stronie serwera
```

**Walidacja:** dla V1 `0 <= upgrade_min <= upgrade_max <= 9`; obie granice są inkluzywne. Jeśli content w przyszłości dostanie inny limit ulepszeń, wymagamy jawnej metadanej `max_upgrade_level` i zaktualizowanego kontraktu. `definition_id` wskazuje **jedną** bazową definicję przedmiotu. Klient nie może wysłać `item_name+9` jako zastępstwa definicji. Brak kategorii / wymagany poziom w obecnym `ItemDefinition` oznacza ukryty filtr do czasu dodania metadanych, a nie zgadywanie.

Bonusy są dopuszczone tylko z katalogu znanych i przeszukiwalnych statystyk; ich typ jednostki (`%`, flat), przedział i precision pochodzą z serwera. Filtrujemy `ItemInstance.affixes` (pojedyncze bonusowe rollsy, **nie** `effective_stats`, zgodnie z sekcją 3). W V1 zduplikowany `stat_id` w różnych regułach oznacza niezależną walidację każdej reguły, bez niejawnego sumowania. Odrzucamy NaN/inf i wartości spoza zakresu; wartości cen i ilości są liczbami całkowitymi bez float.

Wyjście: `offer_id`, `shop_id`, `item_summary` (`definition_id`, prezentacyjna nazwa i faktyczny `upgrade_level`, publiczne bonusy/sockety pod tooltip), `price_yang`, `quantity`, `offer_revision`, `next_cursor`. Dane nie ujawniają kontowych identyfikatorów sprzedawcy ani jego portfela.

## 7.2. Wyszukiwanie katalogu i indeks

`market.catalog.search(text)` daje unikalne `definition_id` z katalogu bazowych przedmiotów, bez rozbijania wersji `+0..+9`. Opcjonalny wpis ręczny pozostaje tekstem aż do wyboru definicji. Wyszukiwanie bazuje na parametryzowanym SQL, indeksach `definition_id`, `upgrade_level`, statusu oferty, ceny, a dla affixów na jawnej indeksowanej projekcji statystyk; **nie** parsujemy wszystkich `affixes_json` przy każdym znaku.

Requesty z CEF mają debounce, `pending/empty/error`, ograniczenie długości, liczby filtrów, paginację/keyset cursor i rate-limit. Jeśli filtr może być nałożony tylko na niektóre typy itemów, UI wyłącza go zgodnie z metadanymi, a backend i tak egzekwuje reguły. W wynikach sortujemy zgodnie z wybranym kluczem i stabilnym `offer_id` dla tie-breaków. Dla stacków pokazujemy `total_price` oraz `unit_price` wyłącznie prezentacyjnie, bez zmiany zasady zakupu całej pozycji.

## 7.3. Otwieranie sklepu z wyniku

`open_shop(shop_id, highlight_offer_id)` zwraca nazwę, status, revision, pełną siatkę, publiczne item summaries i ceny. Klik otwiera **cały sklep**, nie wykonuje zakupu. Highlight jest powiązany z `offer_id` i `item_uid`, a nie pozycją z tabeli (pozycja mogła się zmienić). Jeśli oferta zniknęła, otwórz sklep bez highlight i pokaż `Oferta już niedostępna`, a wyszukiwarkę odśwież; nie zaznaczaj przypadkowego itemu.

### 7.4. Fizyczne stoiska w M1 — etap opcjonalny

Kilka konfigurowalnych stanowisk (np. 4–8) wskazuje istniejące `shop_id`. Rotacja zmienia wyłącznie prezentację w instancji mapy; rynek obejmuje cały World nawet gdy żadna reprezentacja nie stoi w M1. Brak 400 NPC/stoisk, brak ekonomii per-channel.

## 8. Currency Exchange — PoE-style bulk trade

### 8.1. Odrębność od Player Shops

Exchange obsługuje **fungible, dokładnie zdefiniowane aktywa**: Yang i dozwolone stackowalne waluty/materiały. Nie obsługuje mieczy +7 ani itemów z unikalnymi affixami/socketami. Pozycje Currency Exchange **nie zajmują siatki sklepu** i nie mają `shop_id`; mogą działać offline.

System rozróżnia `free_amount` i `reserved_amount`. Wystawienie zlecenia atomowo rezerwuje oferowane aktywa; anulowanie zwraca niewykonaną resztę. Zlecenie nie może opierać się na samym „przyrzeczeniu", że gracz zachowa materiały w inventory.

### 8.2. Parowanie i ceny

- Użytkownik wybiera `want_asset`, `have_asset`, ilość oraz **limit kursu**. Ekran pokazuje obie strony pary; handel ma dostępny arkusz ofert.
- Silnik przechowuje jeden kanoniczny kierunek pary (`base_asset`, `quote_asset`), `side` (`buy|sell`), pozostałą ilość i kurs; `order_id` jest unikalny.
- Zlecenia zgodne cenowo dopasowywane są deterministycznie, **price-time priority** (lepsza cena, następnie wcześniejszy czas / monotoniczny order ID).
- Dozwolone są **partial fills**. Po każdym fill aktualizujemy `remaining_amount`, escrow obu stron, salda i trwały receipt.
- **Bez `float` do cen i rozliczeń.** Kurs reprezentuje para dodatnich liczb całkowitych (licznik/mianownik) albo bezpieczny format fixed-point; rozliczenia są dokładne w całkowitych jednostkach. Rozmiar partii / minimalna ilość musi zapobiegać „ułamkom jednej sztuki"; niewymienialna końcówka pozostaje w zleceniu lub jest anulowana z refundem.
- Porównanie kursów przez mnożenie krzyżowe musi mieć ochronę przed przepełnieniem `int64`. W GDScript **nie zakładamy 128-bit integer**; implementacja wymaga ograniczonych zakresów lub sprawdzonego algorytmu porównania ułamków.
- Wykonanie po kursie oferty spoczywającej (`maker price`). Różnica między limitem kupującego a rzeczywistym wydatkiem wraca z rezerwy do jego salda wolnego.
- Minimalny zakres V1: pary zatwierdzonych materiałów/walut z Yang. Kontrakt danych pozostaje parowy, aby później dopuścić materiał ↔ materiał bez przebudowania orderbooka.
- Prowizja w V1: **brak**; możliwe późniejsze wprowadzenie oddzielnym bilansem/feature flag, nigdy ukrytym zaokrąglaniem.

### 8.3. Materiały jako ledger fungible

Obecnie stackable materiały są `ItemInstance(amount)` w bag/storage. Dla partial fills Exchange powinien używać **kontowego depozytu ilościowego** (`exchange_asset_balances`), nie produkować nowego `ItemInstance` na każdą jednostkową transakcję.

`deposit_to_exchange` zużywa całe/fragmenty legalnych stacków w jednej transakcji (ze zmianą revision i receipts) i kredytuje saldo depozytu. `withdraw_from_exchange` odtwarza stacki w ramach definicji `stack_limit`, z nowymi UID i miejscem w bag/account claim. Tylko itemy o `market_fungible=true`, bez bonusów, ulepszeń i socketów mogą być konwertowane. Własność, ilość i conservation invariant muszą być sprawdzane w SQL oraz testach.

**V1 UX:** depozyt walut jest współdzielony między postaciami konta. Zlecenie zabiera dostępne saldo z depozytu albo Yang; po wykonaniu otrzymane waluty dopisywane są do kontowego depozytu, a Yang bezpośrednio do portfela. Nie wymagamy od kupującego wolnego miejsca w bag przy samym fill. Wypłata materialnych walut do bag jest osobnym krokiem.

### 8.4. Anulowanie, równoległość, offline

Zlecenie żyje po wylogowaniu. `cancel(order_id, revision, command_id)` w transakcji zwraca wyłącznie **niewykonaną** resztę, nigdy część już sprzedaną. Gdy cancellation ściga się z matchingiem, kolejność wyznacza serializowany zapis; drugi aktor widzi revision/stale i nie może wycofać więcej niż pozostało. Zlecenie całkowicie wykonane jest zamykane; historia filli pozostaje dla audytu. Nigdy nie dopuszczamy negative free/reserved.

## 9. Proponowany model tabel (logiczny)

Nazwy i indeksy są punktem wyjścia do migracji, a nie gotowym skryptem do wklejenia bez review istniejących tabel.

```
accounts / world_accounts      account_id, account_name_cached
account_wallets                account_id PK, yang BIGINT CHECK(yang >= 0)
account_shops                  shop_id PK, account_id UNIQUE, name, status, revision
shop_offers                    offer_id PK, shop_id, item_uid UNIQUE, slot, price_yang, revision
shop_escrow                    item_uid PK, account_id, shop_id, offer_id
account_claims                 claim_id PK, account_id, item_uid UNIQUE, status
exchange_asset_balances        account_id, asset_id, free_amount, reserved_amount
exchange_orders                order_id PK, account_id, base_asset, quote_asset,
                               side, limit_ratio, initial_amount, remaining_amount,
                               reserved_amount, status, revision, created_seq
exchange_fills                 fill_id PK, maker_order_id, taker_order_id,
                               base_amount, quote_amount, settled_at
exchange_asset_movements       movement_id PK, asset_id, account_id, delta,
                               reason, command_id
market_commands                account_id, command_id UNIQUE, type, result, created_at
```

**Ważne:** gdy zlecenia Yang trafiają do Exchange, ich rezerwy muszą być częścią tej samej autorytatywnej księgi co `account_wallets`, a nie drugim niezależnym portfelem pozwalającym na podwójny wydatek. Możliwe reprezentacje: `total_yang / reserved_yang` z `available=total-reserved`, albo transfer do jawnego konta escrow wraz z ledgerem. Wybór musi być spójny z obecnym pending-income wallet.

### Niezmienniki bazodanowe

1. `ItemInstance.uid` jest globalnie unikalny i ma dokładnie jedną lokalizację.
2. Dla każdego konta tylko jeden `AccountShop` na World.
3. Jeden item nie może być jednocześnie wystawiony, używany, w magazynie ani w claim.
4. `wallet.available >= 0`, `reserved >= 0`, `free_asset >= 0` po każdym COMMIT.
5. Sumy aktywów w wymianie zachowują się zgodnie z regułą conservation; żadna transakcja nie mintuje aktywów poza zdefiniowanymi source/sink.
6. Jedno `command_id` może skutkować **najwyżej jedną** mutacją; retry zwraca identyczny wynik.
7. Każda zmiana ekonomiczna od razu zapisuje stan trwały; dowolny błąd w połowie transakcji cofa *wszystkie* kroki.
8. Status `paused` wyklucza oferty z wyszukiwania i uniemożliwia zakup, ale nie wypłaca escrow.

## 10. Kontrakty serwer–klient i granice uprawnień

Przykładowe komendy (nazwy do uzgodnienia z istniejącym RPC/CEF bridge):

```
wallet.get
shop.mine.get | shop.mine.rename | shop.mine.pause | shop.mine.resume
shop.offer.list | shop.offer.reprice | shop.offer.move | shop.offer.cancel
market.catalog.search | market.search | market.open_shop | market.purchase
exchange.assets | exchange.book | exchange.deposit | exchange.withdraw
exchange.place_order | exchange.cancel_order | exchange.my_orders
```

Każda mutacja ma `command_id`; mutacje istniejących ofert/items również `expected_revision`. Wymagane odpowiedzi: `ok`, `error_code`, `authoritative_revision`, opcjonalny bezpieczny receipt ID; błędy: `stale`, `unavailable`, `owner`, `funds`, `capacity`, `not_tradeable`, `self_trade`, `rate_limited`, `storage`.

Krytyczne wymagania bezpieczeństwa:

- `account_id` / `character_id` są rozwiązywane **wyłącznie po serwerowej sesji**; klient nie może wybrać konta w RPC.
- Backend nigdy nie ufa podanej przez klienta cenie, saldu, bonusom, amount ani umiejscowieniu bez weryfikacji w transakcji.
- Każda operacja sprawdza pełne uprawnienie oraz World i revision na każdym wywołaniu, nie tylko przy otwarciu okna.
- Ustawione limity długości nazw sklepów, liczby/rozmiaru filtrów, liczby zleceń, paginacji i rate-limit per-account/per-session.
- UI nie ujawnia danych osobowych sprzedawcy, wewnętrznych identyfikatorów Mastera ani prywatnych stanów wallet innych graczy.
- Odpowiedź po timeout może oznaczać **COMMIT bez potwierdzenia**. Client retry używa tego samego `command_id`; nie generuje ponownie intencji, która mogła się wykonać.

# 11. Integracja Core UI / CEF

Nowy frontend korzysta z istniejących `WindowManager`, okien, slotów, item-icons, tooltipów, Inventory drag & drop i mostu native CEF. Logika ekonomii **nigdy** nie trafia do TypeScript: UI renderuje snapshoty i wysyła intencje.

## Ekrany

| Ekran | Funkcje V1 |
|---|---|
| Wallet | Jedno saldo Yang we wszystkich postaciach konta |
| My Shop | Nazwa, status, siatka, Inventory drag/drop, cena, move/reprice/cancel |
| Shop Browser | Nazwa, cała siatka, tooltip, zakup, highlight z wyszukiwarki |
| Market Search | Szkło Handlu UX: katalog bazowych nazw, upgrade od/do (domyślnie +0..+9), filtry rollowanych bonusów AND, kompaktowa lista, tooltip, paginacja, open shop |
| Exchange | Pary, kurs, qty, wolne/zarezerwowane, order book, place/cancel, partial fills |
| Exchange Deposit | Deposit z bag, withdraw do bag lub claim, widok ilości |
| Account Claim | Odbiór anulowanych/pozostałych itemów dowolną postacią |

**Wyniki i podpowiedzi bez pułapek:** `Miecz +0..+9` ma pojedynczy wpis `Miecz` w typeahead; sortowanie wykonuje backend; aktywne filtry są widoczne i możliwe do szybkiego zresetowania. **UX bez pułapek:** po reprice lub buy każda zaintere-

sowana strona dostaje autorytatywny update; tooltips pokazują autentyczne affixy/sockets, nie tylko nazwę definicji; długie nazwy i duże liczby nie rozrywają slotów; UI odblokowuje pending po jednoznacznym statusie lub po sprawdzeniu receipt.

**Dostępność:** docelowo przez interfejs rynku dostępny niezależnie od mapy; można później ograniczyć możliwość edycji sklepu podczas walki bez przebudowy rynku. W V1 nie wymagamy fizycznej tablicy/NPC jako jedynego wejścia.

## 12. Durability, kopie i awarie

W aktualnym `WorldDatabase.open_database` działa WAL z `synchronous=NORMAL`. Przed uruchomieniem handlu między graczami należy **przetestować** `synchronous=FULL`, oszacować koszt zapisu na docelowym dysku i świadomie wybrać politykę trwałości. `NORMAL` przy crashu procesu zwykle zachowuje spójność, lecz utrata zasilania może wycofać ostatni COMMIT; transakcje handlu nie mogą bezrefleksyjnie zakładać gwarancji FULL.

- Backup wykonany przy WAL musi wykonać bezpieczny checkpoint albo użyć SQLite backup API, zgodnie z istniejącą procedurą. Odtwarzanie testujemy, nie tylko liczymy pliki.
- Wstrzymanie/shutdown Worlda blokuje nowe komendy rynku, kończy już rozpoczęte transakcje, zrzuca pending wallet i wykonuje backup.
- Nie ma „przelewu" Yang w osobnym COMMIT po wydaniu itemu; rozliczenie to jedna transakcja SQLite.
- Przy braku bazy lub nieudanym COMMIT nie odsyłamy klientowi sukcesu i nie usuwamy itemu z cache.
- Trwała historia/receipt wystarcza do rozstrzygnięcia niejednoznacznych odpowiedzi po reconnect.
- Zmiana statusu serwera nie może usuwać otwartych ofert ani zleceń Exchange. Restart odczytuje je z SQL i odbudowuje indeks rynku deterministycznie.

## 13. Seasonal → Classic: kontrakt przyszłej migracji

**Cel:** wszystkie postacie i ich trwałe dane oraz majątek przechodzą z Seasonal do Classic, nie nadpisując istniejących postaci ani sald Classic. To jest **proces administracyjny offline z możliwością wznowienia**, nie zwykłe `copy seasonal.db classic.db`.

### Procedura

1. Zamknij sezon: odrzuć nowe operacje ekonomiczne, poczekaj na zakończenie aktywnych transakcji, zrób pełne checkpointy i backupy obu Worldów.
2. Rozlicz Exchange: wyłącz matching, anuluj niewykonane reszty, zwolnij ich escrow i utrwal pełne salda/depozyty. Historia zostaje w sezonowym archiwum.
3. Zamknij oferty sklepów Seasonal i przenieś niesprzedane UIDs do kontowego `claim`. **Nie** łącz dwóch aktywnych sklepów; sklep Classic zostaje niezmieniony.
4. Utwórz manifest transferu dla każdego konta: postacie, item UIDs i lokalizacje, saldo Yang, Account Storage, claim, zasoby Exchange, progresja/questy/skills, pozostałe trwałe dane, sumy kontrolne.
5. W Classic mapuj `player_id` (lokalny dla Worlda) na nowe ID bez kolizji. Powiązane rekordy itemów i pozostałych systemów muszą użyć mapowania. UID itemu pozostaje stabilny, jeśli brak kolizji; wystąpienie kolizji to błąd do ręcznego wyjaśnienia, nie ciche nadpisanie.
6. Dodaj Yang Seasonal do Yang Classic **dokładnie raz**, pod limit. Analogicznie importuj itemy i depozyty, zachowując tożsamość konta. Przedmioty, które nie mieszczą się w inventory/storage, trafiają do kontowego claim.
7. Zweryfikuj globalną i per-account conservation: salda, ilości materiałów, liczby UID, szczegółowe dane przedmiotów. Dopiero po weryfikacji oznacz konto/partię jako przeniesione.
8. Zachowaj sezonowy snapshot read-only i raport migracji; retry jest idempotentny na `season_id + account_id + transfer_version`.

### Decyzje do podjęcia przed rzeczywistym końcem sezonu

- **Limit postaci:** obecny World pozwala na ograniczoną liczbę slotów. Seasonal + istniejący Classic może przekroczyć limit — **nie wolno** usuwać postaci; trzeba przewidzieć dodatkowe/archiwalne sloty.
- **Gildie, rankingi, sezonowe tytuły, questy i waluty eventowe:** określić mapowanie/reguły przed transferem. Deklaracja „przenosimy wszystkie dane" oznacza zachowanie ich tam, gdzie są sensowne, nie zniknięcie niewygodnych kolumn.
- **Itemy/balance niewystępujące w Classic:** jawne reguły konwersji zawartości i wersjonowanie `ItemDefinition`; bez automatycznego niszczenia.
- **Waluty powiązane z sezonem:** osobna decyzja designerska, nie ukryta zmiana przez backend.

W pierwszym wdrożeniu nie musimy pisać narzędzia migracyjnego, ale już teraz potrzebujemy **stabilnego account_id, trwałych UID, historii operacji i oddzielenia rynku od instancji**.

## 14. Testy akceptacyjne

### 14.1. Account-wide Yang

- Dwie postacie konta: grindowanie pierwszą zwiększa saldo widoczne drugiej po reentry; niezależne konto pozostaje nietknięte.
- Migracja sumuje kilka walletów dokładnie raz, także na powtórnym uruchomieniu migratora.
- Tworzenie alternatywnej postaci nie generuje nowego starting Yang.
- NPC Shop, Upgrade, wydatki testowe, ground Yang i disconnect korzystają wyłącznie z account wallet.
- `pending + purchase + offline sale` w różnych kolejnościach kończą z identycznym bilansem; nie ma lost-update ani podwójnego checkpointu.

### 14.2. Player Shops

- Wystawiony UID znika z bag i istnieje tylko w escrow, po relog/restarcie nadal ma te same bonusy, sockety i revision.
- Upgrade/equip/storage/move/claim nie mogą użyć itemu w escrow; SQL nie dopuszcza podwójnej lokalizacji.
- Dwóch kupujących próbuje kupić tę samą ofertę — dokładnie jeden sukces; jeden item i jeden przelew.
- Kupno przez gracza bez miejsca w bag nie pobiera pieniędzy i nie zmienia oferty.
- Self-buy jest zabroniony, także z drugiej postaci tego samego konta.
- Sprzedający offline dostaje Yang; po wejściu do Worlda saldo jest prawidłowe.
- Pauza ukrywa oferty, ale przechowuje escrow; anulowanie zwraca UID do claim.
- Reprice, cancel i purchase w wyścigu dają spójny serializowany rezultat.
- Retry tego samego command ID nie powoduje drugiego zakupu ani utraty Yang.

### 14.3. Search i Exchange

- Wynik wyszukiwania otwiera właściwy sklep i podświetla właściwą ofertę.
- Oferta kupiona po odświeżeniu listy daje `unavailable`, bez błędnego zakupu innego UID.
- `market.catalog.search("miecz")` zwraca bazowe `definition_id` bez 10 rozdzielonych wariantów `+0..+9`; polskie znaki, wielkość liter i nazwy lokalizowane nie zmieniają identyfikatora.
- Domyślny zakres upgrade `+0..+9` zawiera +0, +5 i +9; `+7..+9` obejmuje +7/+8/+9; `+9..+9` dokładnie +9; `+9..+7` jest odrzucony; nieprawidłowe granice nie mogą dotrzeć do SQL.
- Przykładowe bonusy `potwory >= 10%` AND `krytyk >= 5%` zwracają tylko egzemplarze spełniające oba; brak fałszywego trafienia po `effective_stats`, statystyce bazowej, nazwie lub sumowaniu dwóch rolli.

- `stat_id` /operator nieznany serwerowi jest odrzucany; affix roll % i flat nie porównują się w niewłaściwych jednostkach.
- Price filters, upgrade, bonusowe AND i sortowanie są deterministyczne, bez SQL injection lub nieograniczonej paginacji.
- Hover na liście i na slocie sklepu używa wspólnego tooltipa z pełnymi rzeczywistymi bonusami; kliknięcie nie kupuje bezpośrednio z listy.
- Deposit/withdraw stosów materiałów zachowuje łączną ilość, uwzględnia split/merge i stack limits.
- Dwa przeciwstawne zlecenia rozliczają się po maker price; ilość, kurs i refund zgadzają się co do jednostki.
- Partial fills 60/40 dla oferty 100 kończą z `remaining=0`, a otwarta reszta ma poprawną rezerwę.
- Cancel w wyścigu z fill zwraca wyłącznie niewykonaną część.
- Offline order dalej wykonuje się i nie wymaga połączenia właściciela.
- Duże kursy i ilości nie przepełniają `int64`; każda próba wywołująca overflow jest odrzucona *przed* mutacją.
- Restarty i awarie po losowych etapach SQL nie powodują duplikacji assetów ani zgubionych escrow.

### 14.4. Integracja i performance

- `Gateway → Master → World` zweryfikowany identyfikator konta; RPC nie może podszyć się pod cudze konto.
- Testy produkcyjnej ścieżki Native CEF + Godot, nie tylko browser fixtures.
- Benchmark: co najmniej **10 tys. ofert**, wyszukiwanie z filtrami i paginacją oraz równoległe zakupy/depozyty (próg QPS/latency do ustalenia po bazowym pomiarze). Żadne nieograniczone JSON-scan na głównym wątku przy każdym keystroke.
- Test backup→restore na zawierającej aktywne oferty bazie WAL i test recovery po brutalnym zatrzymaniu.
- Pełny regres dotychczasowych `run-yang`, inventory, storage, NPC shop, upgrade i sieciowy smoke.

## 15. Plan implementacji i kryteria wyjścia

Szczegółowy, wykonalny plan dla Codexa (pliki do inspekcji, kolejność PR-ów, testy, ryzyka i definicje *done*) znajduje się w **Mandate of Three — Marketplace Implementation Plan v1.1**. Ten dokument pozostaje źródłem decyzji produktowych i reguł transakcyjnych; osobny plan jest checklistą wykonawczą.

| Paczka / PR | Zakres | Minimalne kryterium wyjścia |
|---|---|---|
| **E0a** | Zaufane `account_id` Master → Gateway → World, mapowanie postaci | Brak możliwości podmiany ownera z klienta; test 2 kont |
| **E0b** | Migracja kontowego Yang i cache RAM, integracja NPC/upgrade/loot | Jedno saldo konta; idempotentny migrator; dotychczasowy suite green |
| **E1a** | SQL custody `bag/storage/shop/claim`, migracje, receipts | Każdy UID ma dokładnie jedną lokalizację, także po rollback |
| **E1b** | My Shop: siatka, nazwa, ceny, reprice/cancel/pause, trwały escrow | Oferta i układ żyją po restart; działa edycja z CEF |
| **E2** | Atomowy zakup Player Shop + offline seller + idempotency | 2 kupujących na 1 ofertę → 1 sukces, żaden Yang/UID nie ginie |
| **E3a** | Backend Market Search i `catalog.search`: `definition_id`, `+0..+9`, AND affix | Testy indeksu i granic; bez N wariantów w type-ahead |

| Paczka / PR | Zakres | Minimalne kryterium wyjścia |
|---|---|---|
| **E3b** | CEF Search: kompaktowa tabela, tooltip, `open_shop + highlight` | Pełny flow wyszukiwarka → sklep → zakup w native CEF |
| **E4** | Kontowy ledger aktywów Exchange, deposit/withdraw, free/reserved | Conservation dla split/merge, ponowień i restartów |
| **E5** | Orderbook, matching, partial fills, kursy exact, UI PoE-style | Deterministyczne fill/cancel, brak overflow, test odbudowy |
| **E6 (opcjonalnie)** | Kilka rotujących stoisk w M1 | Tylko warstwa prezentacji; marketplace nadal globalny |
| **S (później)** | Przeniesienie Seasonal → Classic | Manifest, mapowanie postaci, idempotentny import bez strat |

**Ważna kolejność:** E3a można prototypować na sztucznych ofertach po E1a, ale pełne E3b musi używać prawdziwego E2. Nie łącz E0–E5 w jeden PR; na końcu każdej paczki regres dotychczasowej ekonomii, a od E2 również próba awaryjna/rollback. **Nie** implementować kanałów, PostgreSQL, makr UI ani abstrakcyjnego ORM „przy okazji".

### Kryteria gotowości przed pierwszym publicznym marketem

- Żadnych znanych duplicate spend / duplicate item / stale wallet ani podwójnych eventów po reconnect.
- Backup, restart i odzyskiwanie ofert sprawdzone na prawdziwej bazie SQLite i WAL.
- Zmiana konta, postaci czy przyszłej instancji nie daje dostępu do cudzego sklepu ani portfela.
- Transakcje mają receipts i bezpieczne retries.
- Market Search poprawnie filtruje bazową definicję + niezależny upgrade OD/DO + AND bonusów instancji.
- Native CEF pokazuje kompaktowe wyniki, tooltip i właściwy sklep z podświetleniem.
- CI: testy istniejące + testy rynku + benchmark wyszukiwania 10 tys. ofert.
- Polityki `WAL synchronous`, limity, maksymalne Yang i handlowalność udokumentowane i przetestowane.

## 16. Pytania designerskie, które mogą poczekać

1. Ostateczna pojemność sklepu (32 sloty to tylko propozycja).
2. Czy docelowo Market UI jest dostępny wszędzie, czy edycja wymaga bezpiecznej strefy?
3. Jakie materiały oprócz Yang wchodzą do Exchange i które pary mają być dostępne w pierwszym sezonie?
4. Czy chcemy prowizje, limity zleceń i aktywne sinki Yang? W V1: brak prowizji.
5. Czy w przyszłości wybierzemy ręczne Channels czy automatyczne Layers? **Bez wpływu na obecny rynek.**
6. Dokładna polityka przenoszenia danych specyficznych dla sezonu/gildii do Classic.

## 17. Referencje repozytorium i UX

Przejrzane pliki na SHA wskazanym w metadanych. Główna referencja UX: Metin2 — Szkło Handlu, polska wiki.

- `WorldDatabase`
- `WalletStoreSqlite`

- `ItemStoreSqlite`
- `AccountStorageSqlite`
- `WorldSchema`
- `ShopStoreSqlite`
- `UpgradeStoreSqlite`
- `WorldStoreSqlite`
- `CurrencyEndpoint3D`
- `AccountResource`
- `Core UI`
- `Persistence Policy`

---

**Werdykt implementacyjny:** zaczynamy od E0 (account-wide Yang), bo wszystkie późniejsze zakupy i zlecenia muszą używać docelowej definicji portfela. Pierwszy Player Shop i Currency Exchange mogą później zostać dodane bez zmiany fundamentalnego modelu danych Worlda.